

LAPORAN PRAKTIKUM STRUKTUR DATA

“Single Linked List”

Disusun Oleh:

Aryahiyahul Fikra

2511532026

Dosen Pengampu:

Dr. Wahyudi S.T, M.T

Asisten Pembimbing:

Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

Daftar Isi

Daftar Isi	2
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Tujuan Praktikum	3
BAB II.....	4
PEMBAHASAN	4
2.1 Pengertian Single Linked List	4
2.2 Program NodeSLL_2511532026	4
2.3 Program TambahSLL_2511532026	4
2.4 Program HapusSLL_2511532026	6
2.5 Program PencarianSLL_2511532026	8
BAB III	9
PENUTUP	9
3.1 Kesimpulan	9
TUGAS SLL	10
Program Pasien_2511532026	10
Program RumahSakit_2511532026	11
Output Program	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara untuk menyimpan dan mengelola data agar dapat digunakan secara efisien. Salah satu struktur data yang sering digunakan adalah *Single Linked List* (SLL). Single Linked List adalah struktur data linear yang terdiri dari beberapa node, dimana setiap node memiliki data dan pointer yang menunjuk ke node berikutnya.

Penggunaan Single Linked List memiliki kelebihan dibanding array, yaitu ukuran data dapat berubah secara dinamis tanpa harus menentukan kapasitas sejak awal. Oleh karena itu, pemahaman mengenai operasi-operasi pada Single Linked List sangat penting dalam pemrograman.

Pada praktikum ini dilakukan implementasi beberapa operasi dasar Single Linked List menggunakan bahasa pemrograman Java, meliputi penambahan node, penghapusan node, dan pencarian data pada linked list. Dengan praktikum ini diharapkan mahasiswa dapat memahami konsep dasar serta cara kerja struktur data Single Linked List.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Single Linked List.
2. Memahami struktur node pada Single Linked List.
3. Mengimplementasikan operasi penambahan node pada linked list.
4. Mengimplementasikan operasi penghapusan node pada linked list.
5. Mengimplementasikan proses pencarian data pada linked list menggunakan Java.

BAB II PEMBAHASAN

2.1 Pengertian Single Linked List

Single Linked List adalah struktur data linear yang terdiri dari kumpulan node. Setiap node memiliki dua bagian utama, yaitu data dan pointer yang menunjuk ke node berikutnya. Linked List bersifat dinamis sehingga penambahan maupun penghapusan data dapat dilakukan dengan mudah.

2.2 Program NodeSLL_2511532026

Program ini dibutuhkan untuk membuat struktur Node pada Single Linked List. Node terdiri dari variable data dan pointer next yang menunjuk ke node berikutnya.

```
1 package pekan5_2511532026;
2
3 public class NodeSLL_2511532026 {
4     //node bagian data
5     int data_2026;
6     //pointer ke node berikutnya
7     NodeSLL_2511532026 next_2026;
8     //konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511532026(int data_2026) {
10         this.data_2026 =data_2026;
11         this.next_2026 =null;
12
13     }
14
15 }
16 }
```

Penjelasan:

- data_2026 digunakan untuk menyimpan nilai data.
- next_2026 digunakan sebagai pointer ke node berikutnya.
- Constructor digunakan untuk menginisialisasi data node.

2.3 Program TambahSLL_2511532026

Program ini digunakan untuk menambahkan node pada Single Linked List. Operasi yang dilakukan yaitu:

1. Menambah node di depan (*insertAtFront*)
2. Menambah node di belakang (*insertAtEnd*)
3. Menambah node pada posisi tertentu (*insertPos*)

Kode Program:

```
1 package pekan5_2511532026;
2
3 public class TambahSLL_2511532026 {
4     public static NodeSLL_2511532026 insertAtFront_2026(NodeSLL_2511532026 head, int value) {
5         NodeSLL_2511532026 new_node = new NodeSLL_2511532026(value);
6         new_node.next_2026 = head;
7         return new_node;
8     }
9
10    // fungsi menambahkan node di akhir SLL
11    public static NodeSLL_2511532026 insertAtEnd_2026(NodeSLL_2511532026 head, int value) {
12        // buat sebuah node dengan sebuah nilai
13        NodeSLL_2511532026 newNode = new NodeSLL_2511532026(value);
14
15        // jika list kosong maka node jadi head
16        if (head == null) {
17            return newNode;
18        }
19
20        // simpan head ke variabel sementara
21        NodeSLL_2511532026 last = head;
22
23        // telusuri ke node akhir
24        while (last.next_2026 != null) {
25            last = last.next_2026;
26        }
27
28        // ubah pointer
29        last.next_2026 = newNode;
30        return head;
31    }
32
33    static NodeSLL_2511532026 GetNode_2026(int data) {
34        return new NodeSLL_2511532026(data);
35    }
36    static NodeSLL_2511532026 insertPos_2026(NodeSLL_2511532026 headNode, int position, int value) {
37        NodeSLL_2511532026 head = headNode;
38
39        if (position < 1)
40            System.out.print("Invalid position");
41
42        if (position == 1) {
43            NodeSLL_2511532026 new_node = new NodeSLL_2511532026(value);
44            new_node.next_2026 = head;
45            return new_node;
46        } else {
47            while (position-- != 0) {
48                if (position == 1) {
49                    NodeSLL_2511532026 newNode = GetNode_2026(value);
50                    newNode.next_2026 = headNode.next_2026;
51                    headNode.next_2026 = newNode;
52                    break;
53                }
54                headNode = headNode.next_2026;
55            }
56
57            if (position != 1)
58                System.out.print("Posisi di luar jangkauan");
59        }
60        return head;
61    }
62
63    public static void printList_2026(NodeSLL_2511532026 head) {
64        NodeSLL_2511532026 curr = head;
65
66        while (curr.next_2026 != null) {
67            System.out.print(curr.data_2026 + "-->");
68            curr = curr.next_2026;
69        }
70
71        if (curr.next_2026 == null) {
72            System.out.print(curr.data_2026);
73        }
74
75        System.out.println();
76    }
77
78    public static void main(String[] args) {
79        // buat linked list 2->3->5->6
80        NodeSLL_2511532026 head_2026 = new NodeSLL_2511532026(2);
81        head_2026.next_2026 = new NodeSLL_2511532026(3);
82        head_2026.next_2026.next_2026 = new NodeSLL_2511532026(5);
83        head_2026.next_2026.next_2026.next_2026 = new NodeSLL_2511532026(6);
84
85        // cetak list asal
86        System.out.print("Senarai berantai awal:");
87        printList_2026(head_2026);
88
89        // tambahkan node baru di depan
90        System.out.print("Tambah 1 simpul di depan: ");
91
92        int data_2026 = 1;
93        head_2026 = insertAtFront_2026(head_2026, data_2026);
94
95        // cetak update list
96        printList_2026(head_2026);
97        //tambahkan node baru dibelakang
98        System.out.print("Tambah 1 simpul di belakang: ");
99        int data2_2026 = 7;
100        head_2026 = insertAtEnd_2026(head_2026, data2_2026);
101
102        // cetak update list
103        printList_2026(head_2026);
104
105        // tambah node di posisi tertentu
106        System.out.print("Tambah 1 simpul ke data 4: ");
107        int data3_2026 = 4;
108        int pos_2026 = 4;
109        head_2026 = insertPos_2026(head_2026, pos_2026, data3_2026);
110
111        // cetak update list
112        printList_2026(head_2026);
113    }
```

Penjelasan Program

- Method insertAtFront_2026() digunakan untuk menambahkan node di awal linked list.
- Method insertAtEnd_2026() digunakan untuk menambahkan node di akhir linked list.
- Method insertPos_2026() digunakan untuk menyisipkan node pada posisi tertentu.

- Method `printList_2026()` digunakan untuk menampilkan isi linked list.

Output Program:

```
<terminated> TambahSLL_2511532026 [Java Application] C:\Users\ryhiyhu\Do
Senarai berantai awal:2-->3-->5-->6
Tambah 1 simpul di depan: 1-->2-->3-->5-->6
Tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
Tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

2.4 Program HapusSLL_2511532026

Program ini digunakan untuk menghapus node pada Single Linked List. Operasi yang dilakukan yaitu:

1. Menghapus node pertama (*deleteHead*)
2. Menghapus node terakhir (*removeLastNode*)
3. Menghapus node pada posisi tertentu (*deleteNode*)

Kode Program:

```
1 package pekan5_2511532026;
2
3 public class HapusSLL_2511532026 {
4     public static NodeSLL_2511532026 deleteHead_2026(NodeSLL_2511532026 head_2026) {
5         if (head_2026 == null)
6             return null;
7         head_2026 = head_2026.next_2026;
8         return head_2026;
9     }
10 }
11 public static NodeSLL_2511532026 removeLastNode_2026(NodeSLL_2511532026 head_2026) {
12     if (head_2026 == null) {
13         return null;
14     }
15     if (head_2026.next_2026 == null) {
16         return null;
17     }
18     NodeSLL_2511532026 secondlast_2026 = head_2026;
19     while (secondlast_2026.next_2026.next_2026 != null) {
20         secondlast_2026 = secondlast_2026.next_2026;
21     }
22     secondlast_2026.next_2026 = null;
23     return head_2026;
24 }
25 // fungsi menghapus node di posisi tertentu
26 public static NodeSLL_2511532026 deleteNode_2026(NodeSLL_2511532026 head, int position) {
27     NodeSLL_2511532026 temp_2026 = head;
28     NodeSLL_2511532026 prev_2026 = null;
29
30     // jika linked list null
31     if (temp_2026 == null)
32         return head;
33
34     // kasus 1: head dihapus
35     if (position == 1) {
36         head = temp_2026.next_2026;
37         return head;
38     }
39
40     // kasus 2: menghapus node di tengah
41     // telusuri ke node yang dihapus
42     for (int i = 1; temp_2026 != null && i < position; i++) {
43         prev_2026 = temp_2026;
44     }
45 }
```

```

46         temp_2026 = temp_2026.next_2026;
47     }
48     // jika ditemukan, hapus node
49     if (temp_2026 != null)
50         prev_2026.next_2026 = temp_2026.next_2026;
51     else
52         System.out.println("Data tidak ada");
53
54     return head;
55 }
56
57 // fungsi mencetak SLL
58 public static void printList_2026(NodeSLL_2511532026 head) {
59     NodeSLL_2511532026 curr_2026 = head;
60
61     while (curr_2026.next_2026 != null) {
62         System.out.print(curr_2026.data_2026 + "-->");
63         curr_2026 = curr_2026.next_2026;
64     }
65
66     if (curr_2026.next_2026 == null) {
67         System.out.print(curr_2026.data_2026);
68     }
69
70     System.out.println();
71
72 }
73
74 // kelas main
75 public static void main(String[] args) {
76     // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
77     NodeSLL_2511532026 head_2026 = new NodeSLL_2511532026(1);
78     head_2026.next_2026 = new NodeSLL_2511532026(2);
79     head_2026.next_2026.next_2026 = new NodeSLL_2511532026(3);
80     head_2026.next_2026.next_2026.next_2026 = new NodeSLL_2511532026(4);
81     head_2026.next_2026.next_2026.next_2026.next_2026 = new NodeSLL_2511532026(5);
82     head_2026.next_2026.next_2026.next_2026.next_2026.next_2026 = new NodeSLL_2511532026(6);
83
84     // cetak list awal
85     System.out.println("List awal: ");
86     printList_2026(head_2026);
87
88     // hapus head
89     head_2026 = deleteHead_2026(head_2026);
90     System.out.println("List setelah head dihapus: ");

```

```

91     printList_2026(head_2026);
92
93     // hapus node terakhir
94     head_2026 = removeLastNode_2026(head_2026);
95     System.out.println("List setelah simpul terakhir di hapus: ");
96     printList_2026(head_2026);
97
98     // Deleting node at position 2
99     int position = 2;
100     head_2026 = deleteNode_2026(head_2026, position);
101
102     // Print list after deletion
103     System.out.println("List setelah posisi 2 dihapus: ");
104     printList_2026(head_2026);
105 }
106
107 }

```

```

<terminated> HapusSLL_2511532026 [Java Application]
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

```

Penjelasan Program

- Method deleteHead_2026() digunakan untuk menghapus node pertama.
- Method removeLastNode_2026() digunakan untuk menghapus node terakhir.
- Method deleteNode_2026() digunakan untuk menghapus node pada posisi tertentu.
- Method printList_2026() digunakan untuk menampilkan isi linked list setelah penghapusan.

2.5 Program PencarianSLL_2511532026

Program ini digunakan untuk melakukan pencarian data pada Single Linked List.

Kode Program:

```
1 package pekan5_2511532026;
2
3 public class PencarianSLL_2511532026 {
4     static boolean searchKey_2026 (NodeSLL_2511532026 head_2026, int key_2026) {
5         NodeSLL_2511532026 curr_2026 = head_2026;
6         while (curr_2026 != null) {
7             if (curr_2026.data_2026 == key_2026)
8                 return true;
9             curr_2026 = curr_2026.next_2026;
10        }
11        return false;
12    }
13    public static void traversal_2026 (NodeSLL_2511532026 head) {
14        NodeSLL_2511532026 curr_2026 = head;
15
16        while (curr_2026 != null) {
17            System.out.println(" " + curr_2026.data_2026);
18            curr_2026 = curr_2026.next_2026;
19        }
20        System.out.println();
21    }
22    public static void main(String[] args) {
23        NodeSLL_2511532026 head_2026 = new NodeSLL_2511532026(14);
24        head_2026.next_2026 = new NodeSLL_2511532026(21);
25        head_2026.next_2026.next_2026 = new NodeSLL_2511532026(13);
26        head_2026.next_2026.next_2026.next_2026 = new NodeSLL_2511532026(30);
27        head_2026.next_2026.next_2026.next_2026.next_2026 = new NodeSLL_2511532026(10);
28
29        System.out.print("Penelusuran SLL : ");
30        traversal_2026(head_2026);
31
32        // data yang akan dicari
33        int key = 30;
34        System.out.print("cari data " + key + " : ");
35
36        if (searchKey_2026(head_2026, key))
37            System.out.println("ketemu");
38        else
39            System.out.println("tidak ada");
40    }
41 }
42 }
```

Penjelasan Program

- Method searchKey_2026() digunakan untuk mencari data tertentu pada linked list.
- Method traversal_2026() digunakan untuk menampilkan seluruh isi linked list.
- Program akan mengecek apakah data yang dicari tersedia atau tidak.

Output:

```
<terminated> PencarianSLL_2511532026
Penelusuran SLL : 14
21
13
30
10

cari data 30 : ketemu
```


BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data linear yang terdiri dari node-node yang saling terhubung. Pada praktikum ini berhasil diimplementasikan operasi dasar Single Linked List menggunakan Java, yaitu penambahan node, penghapusan node, dan pencarian data. Dengan memahami konsep linked list, mahasiswa dapat memahami cara pengelolaan data dinamis dalam struktur data.

TUGAS SLL

Program Pasien_2511532026

Kode Program:

```
1 package pekan5_2511532026;
2 public class Pasien_2511532026 {
3     private String namaPasien_2026;
4     private String keluhan_2026;
5     private int nomorAntrian_2026;
6     private Pasien_2511532026 next_2026;
7
8     public Pasien_2511532026(String namaPasien_2026, String keluhan_2026, int nomorAntrian_2026) {
9         this.namaPasien_2026 = namaPasien_2026;
10        this.keluhan_2026 = keluhan_2026;
11        this.nomorAntrian_2026 = nomorAntrian_2026;
12        this.next_2026 = null;
13    }
14
15    public String getNamaPasien_2026() {
16        return namaPasien_2026;
17    }
18
19    public String getKeluhan_2026() {
20        return keluhan_2026;
21    }
22
23    public int getNomorAntrian_2026() {
24        return nomorAntrian_2026;
25    }
26
27    public Pasien_2511532026 getNext_2026() {
28        return next_2026;
29    }
30
31    public void setNamaPasien_2026(String namaPasien_2026) {
32        this.namaPasien_2026 = namaPasien_2026;
33    }
34
35    public void setKeluhan_2026(String keluhan_2026) {
36        this.keluhan_2026 = keluhan_2026;
37    }
38
39    public void setNomorAntrian_2026(int nomorAntrian_2026) {
40        this.nomorAntrian_2026 = nomorAntrian_2026;
41    }
42
43    public void setNext_2026(Pasien_2511532026 next_2026) {
44        this.next_2026 = next_2026;
45    }
```

Pasien_2511532026 berfungsi sebagai **representasi satu node** pada struktur **single linked list**. Artinya, setiap objek yang dibuat dari kelas ini mewakili **satu data pasien** di dalam antrian rumah sakit.

Di dalam kelas ini terdapat empat atribut, yaitu namaPasien_2026 untuk menyimpan nama pasien, keluhan_2026 untuk menyimpan keluhan atau penyakit, nomorAntrian_2026 untuk menyimpan urutan antrian, dan next_2026 yang berfungsi sebagai **pointer** atau penghubung ke node pasien berikutnya.

Saat sebuah objek Pasien_2511532026 dibuat melalui constructor, program langsung mengisi nama pasien, keluhan, dan nomor antrian sesuai data yang dimasukkan. Sementara itu, next_2026 diisi null, yang berarti node tersebut belum terhubung dengan node lain. Nantinya, ketika ada pasien baru ditambahkan ke antrian, nilai next_2026 akan diubah agar node saling terhubung membentuk linked list.

Kelas ini juga menyediakan **getter** dan **setter**. Getter digunakan untuk mengambil informasi pasien, misalnya saat data ditampilkan atau dicari. Setter digunakan untuk mengubah data pasien serta mengatur pointer next_2026. Dengan adanya atribut dan method tersebut, Pasien_2511532026 menjadi komponen dasar yang menyimpan data sekaligus membentuk hubungan antar node pada antrian rumah sakit.

Program RumahSakit_2511532026

Kode Program:

```
1 package pekans_2511532026;
2 import java.util.Scanner;
3
4 public class RumahSakit_2511532026 {
5     private Pasien_2511532026 head_2026;
6     private int counter_2026;
7
8     public RumahSakit_2511532026() {
9         head_2026 = null;
10        counter_2026 = 0;
11    }
12
13    // Insert at tail
14    public void daftarkanPasien_2026(String namaPasien_2026, String keluhan_2026) {
15        counter_2026++;
16        Pasien_2511532026 pasienBaru_2026 =
17            new Pasien_2511532026(namaPasien_2026, keluhan_2026, counter_2026);
18
19        if (head_2026 == null) {
20            head_2026 = pasienBaru_2026;
21        } else {
22            Pasien_2511532026 bantu_2026 = head_2026;
23            while (bantu_2026.getNext_2026() != null) {
24                bantu_2026 = bantu_2026.getNext_2026();
25            }
26            bantu_2026.setNext_2026(pasienBaru_2026);
27        }
28
29        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_2026);
30    }
31
32    // Delete head
33    public void panggilPasien_2026() {
34        if (head_2026 == null) {
35            System.out.println("Antrian kosong.");
36            return;
37        }
38
39        System.out.println("Pasien dipanggil:");
40        System.out.println("Nomor Antrian : " + head_2026.getNomorAntrian_2026());
41        System.out.println("Nama : " + head_2026.getNamaPasien_2026());
42        System.out.println("Keluhan : " + head_2026.getKeluhan_2026());
43
44        head_2026 = head_2026.getNext_2026();
45    }
```

```
47 // Display
48 public void tampilkanAntrian_2026() {
49     if (head_2026 == null) {
50         System.out.println("Antrian kosong.");
51         return;
52     }
53
54     Pasien_2511532026 bantu_2026 = head_2026;
55     int posisi_2026 = 1;
56
57     System.out.println("=== Daftar Antrian Pasien ===");
58
59     while (bantu_2026 != null) {
60         System.out.println("Posisi : " + posisi_2026);
61         System.out.println("Nomor Antrian : " + bantu_2026.getNomorAntrian_2026());
62         System.out.println("Nama : " + bantu_2026.getNamaPasien_2026());
63         System.out.println("Keluhan : " + bantu_2026.getKeluhan_2026());
64         System.out.println("-----");
65
66         bantu_2026 = bantu_2026.getNext_2026();
67         posisi_2026++;
68     }
69 }
70
71 // Search
72 public void cariPasien_2026(String namaCari_2026) {
73     if (head_2026 == null) {
74         System.out.println("Antrian kosong.");
75         return;
76     }
77
78     Pasien_2511532026 bantu_2026 = head_2026;
79     boolean ditemukan_2026 = false;
80
81     while (bantu_2026 != null) {
82         if (bantu_2026.getNamaPasien_2026().equalsIgnoreCase(namaCari_2026)) {
83             System.out.println("Pasien ditemukan:");
84             System.out.println("Nomor Antrian : " + bantu_2026.getNomorAntrian_2026());
85             System.out.println("Nama : " + bantu_2026.getNamaPasien_2026());
86             System.out.println("Keluhan : " + bantu_2026.getKeluhan_2026());
87             ditemukan_2026 = true;
88             break;
89         }
90         bantu_2026 = bantu_2026.getNext_2026();
91     }
```

```

93     if (!ditemukan_2026) {
94         System.out.println("Pasien tidak ditemukan.");
95     }
96 }
97
98 // Status
99 public void cekStatusAntrian_2026() {
100     if (head_2026 == null) {
101         System.out.println("Antrian kosong.");
102         return;
103     }
104
105     int jumlah_2026 = 0;
106     Pasien_2511532026 bantu_2026 = head_2026;
107
108     while (bantu_2026 != null) {
109         jumlah_2026++;
110         bantu_2026 = bantu_2026.getNext_2026();
111     }
112
113     System.out.println("Jumlah pasien dalam antrian : " + jumlah_2026);
114     System.out.println("Pasien terdepan : " + head_2026.getNamaPasien_2026());
115     System.out.println("Nomor antrian : " + head_2026.getNomorAntrian_2026());
116 }
117
118 public static void main(String[] args) {
119     Scanner input_2026 = new Scanner(System.in);
120     RumahSakit_2511532026 rs_2026 = new RumahSakit_2511532026();
121
122     int pilihan_2026;
123
124     do {
125         System.out.println("\n=== Antrian Rumah Sakit NIM: 2511532026 ===");
126         System.out.println("1. Daftarkan Pasien (Insert)");
127         System.out.println("2. Panggil Pasien (Delete Head)");
128         System.out.println("3. Tampilkan Antrian (Display)");
129         System.out.println("4. Cari Pasien (Search)");
130         System.out.println("5. Cek Status Antrian");
131         System.out.println("6. Keluar");
132         System.out.print("Pilihan: ");
133
134         pilihan_2026 = input_2026.nextInt();
135         input_2026.nextLine();
136
137         switch (pilihan_2026) {

```

```

138             case 1:
139                 System.out.print("Masukkan Nama Pasien : ");
140                 String nama_2026 = input_2026.nextLine();
141
142                 System.out.print("Masukkan Keluhan : ");
143                 String keluhan_2026 = input_2026.nextLine();
144
145                 rs_2026.daftarkanPasien_2026(nama_2026, keluhan_2026);
146                 break;
147
148             case 2:
149                 rs_2026.panggilPasien_2026();
150                 break;
151
152             case 3:
153                 rs_2026.tampilkanAntrian_2026();
154                 break;
155
156             case 4:
157                 System.out.print("Masukkan nama pasien yang dicari: ");
158                 String cari_2026 = input_2026.nextLine();
159                 rs_2026.cariPasien_2026(cari_2026);
160                 break;
161
162             case 5:
163                 rs_2026.cekStatusAntrian_2026();
164                 break;
165
166             case 6:
167                 System.out.println("Program selesai.");
168                 break;
169
170             default:
171                 System.out.println("Pilihan tidak valid.");
172         }
173     } while (pilihan_2026 != 6);
174
175     input_2026.close();
176 }
177 }
178 }

```

RumahSakit_2511532026 berfungsi untuk **mengelola seluruh antrian pasien**. Kelas ini mengatur proses penambahan pasien, pemanggilan pasien, pencarian data, penampilan antrian, dan pengecekan status antrian.

Di dalam kelas ini terdapat dua atribut utama, yaitu head_2026 dan counter_2026. Variabel head_2026 digunakan sebagai pointer yang menunjuk ke **node pertama** pada linked list. Jika head_2026 bernilai null, berarti antrian masih kosong. Sedangkan counter_2026 digunakan untuk menyimpan nomor antrian terakhir, sehingga setiap pasien baru yang didaftarkan akan memperoleh nomor secara otomatis.

Method daftarkanPasien_2026() digunakan untuk menambahkan pasien baru ke **bagian akhir linked list** (*insert at tail*). Jika antrian kosong, node baru langsung menjadi head. Jika antrian sudah berisi data, program menelusuri node sampai node terakhir, lalu menghubungkan node terakhir tersebut ke node baru melalui pointer next_2026.

Method panggilPasien_2026() digunakan untuk memanggil pasien yang berada di urutan paling depan. Data pasien pertama ditampilkan, kemudian head_2026 digeser ke node berikutnya. Dengan cara ini, node pertama terhapus dari antrian. Proses ini sesuai dengan prinsip **FIFO (First In, First Out)**.

Method tampilkanAntrian_2026() berfungsi untuk menampilkan seluruh data pasien. Program melakukan penelusuran mulai dari head_2026 hingga node terakhir (null). Setiap node menampilkan nomor antrian, nama pasien, keluhan, dan posisi pasien di dalam antrian.

Method cariPasien_2026() digunakan untuk mencari data pasien berdasarkan nama. Pencarian dilakukan dengan menelusuri node satu per satu. Perbandingan nama menggunakan equalsIgnoreCase(), sehingga huruf besar dan kecil dianggap sama.

Method cekStatusAntrian_2026() berfungsi untuk menampilkan kondisi antrian. Program menghitung jumlah seluruh pasien yang sedang menunggu, kemudian menampilkan informasi pasien yang berada di posisi terdepan.

Secara keseluruhan, RumahSakit_2511532026 berperan sebagai **pengendali linked list**, sedangkan Pasien_2511532026 berperan sebagai **node penyimpan data**. Keduanya bekerja bersama untuk membentuk sistem antrian rumah sakit yang dinamis.

Output Program

```
RumahSakit_2511532026 [Java Application] C:\Users\ryhiyhu\Downloa
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Ary
Masukkan Keluhan      : ganteng
Pasien berhasil didaftarkan! Nomor Antrian: 1
```